UNIVERSIDAD TECNICA FEDERICO SANTA MARIA

DEPARTAMENTO DE INFORMÁTICA

CCTVal

OCEANS

Funded by the European Union

SOCHIAS
sociedad chilena de astronomía

Machine Learning Introduction

Stellar parameter estimation using ML

Raquel Pezoa Rivera – Bastián Wohlwend

Departamento de Informática · Centro Científico Tecnológico de Valparaíso

Universidad Técnica Federico Santa María

XXI SOCHIAS Annual Meeting, October 28th, 2026 – Valparaíso, Chile

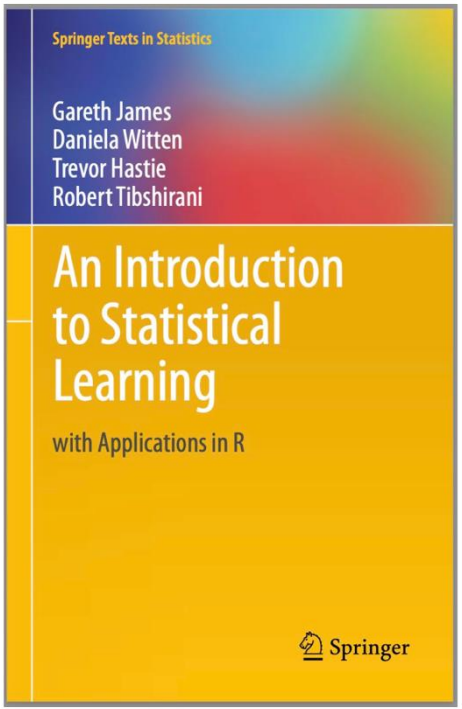
Outline

- ML definitions and basic concepts
- Deep learning
- Bayesian NN

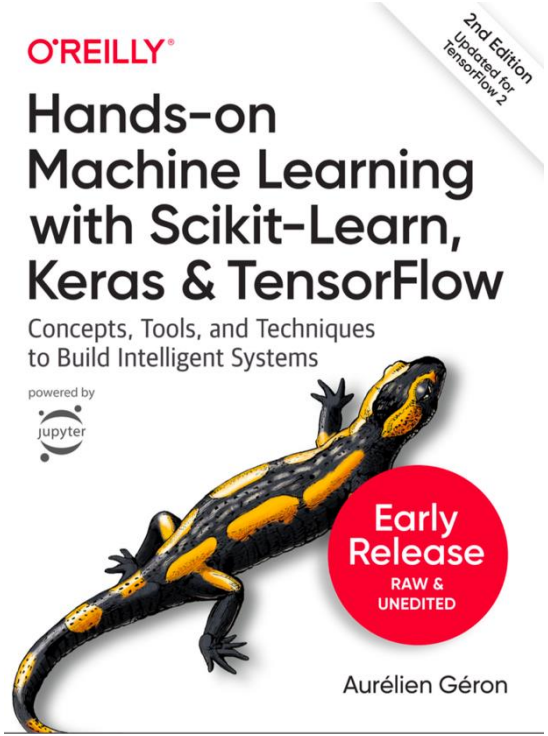


ML Definitions

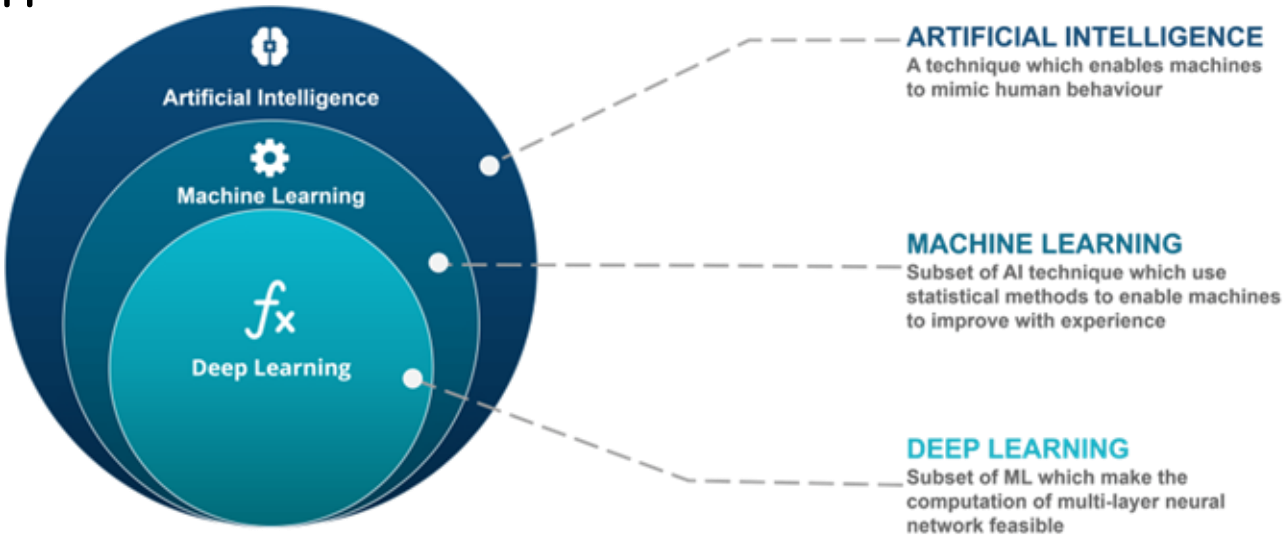
Some interesting books



https://www.statlearning.com/



Introduction



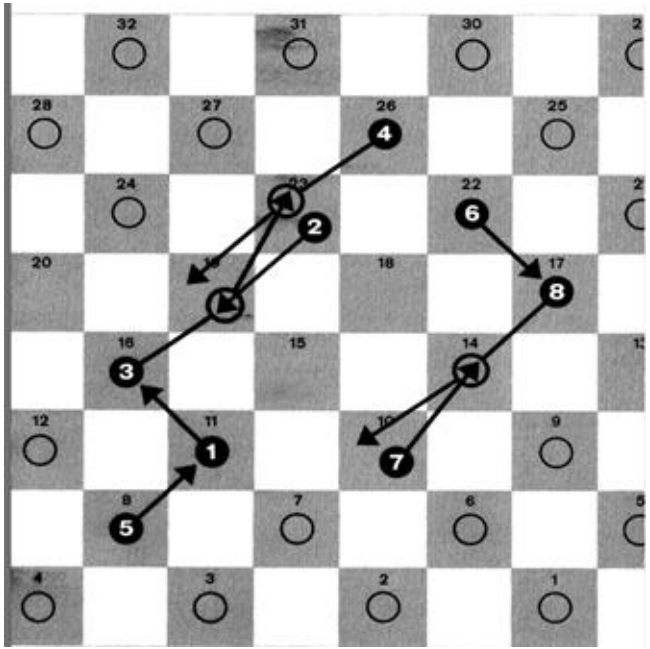
https://cognitive.la/blog/machine-learning-vs-deep-learning

Machine learning (ML) → development of computational algorithms that learn and improve automatically through experience.

Some definitions

- Arthur Samuel (1959): ML is the field that *"gives computers the ability to learn without being explicitly programmed"* [1] → Arthur Samuel coined the term "machine learning".

In ML we don't manually write rules or instructions for the system to follow step by step.



[1] A. L. Samuel, "Some studies in machine learning using the game of checkers," in *IBM Journal of Research and Development*, vol. 44, no. 1.2, pp. 206-226, Jan. 2000, doi: 10.1147/rd.441.0206.https://ieeexplore.ieee.org/document/5391906 .

Some defintions

- "*Machine Learning is the science (and art) of programming computers so they can learn from data*". [3]

- *"Machine learning (ML) is a sub-branch of AI that focuses on teaching computers how to learn without the need to be programmed for specific tasks* ".[4]

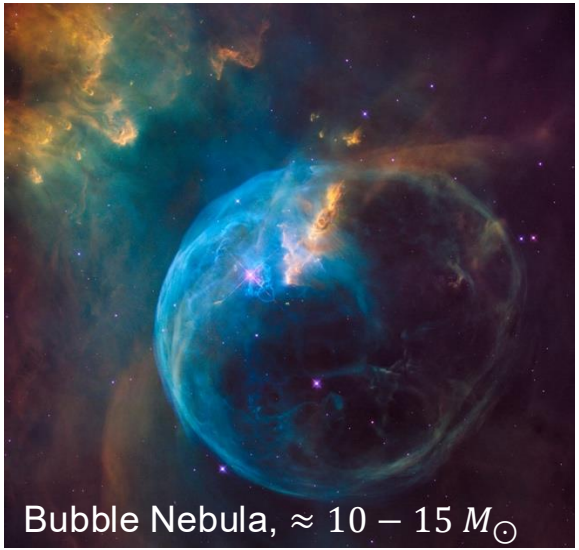
[3] Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow. Concepts, Tools, and Techniques to Build Intelligent Systems. Aurélien Géron. 2019.

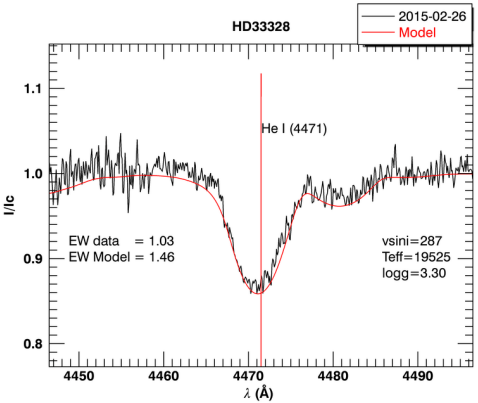
[4] Deep Learning with Keras. Antonio Gulli, Sujit Pal. 2017.

Basic concepts

Main Idea

We have a problem/task:
"predict stellar parameters"





(*)



We have data

We need to find the relationships of input and output variables $f(x) = y$ (with good performance!)

(*) Arcos, C et al. (2017). Stellar parameters and Hα line profile variability of Be stars in the BeSOS survey. Monthly Notices of the Royal Astronomical Society. 474. 10.1093/mnras/stx3075.

Main Idea

- **To predict** an output value from input data
- And we have data:

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

Feature, input variable — Label, output variables, target

- A (true) function $Y = f(X) + \epsilon$ and an estimate $\hat{Y} \approx \hat{f}(X)$
- We use a **loss function** to measure the goodness of the approximation → **optimization problem**

ML Concepts

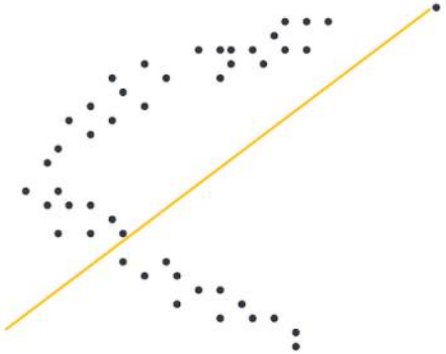
- A main challenge in ML → the algorithm must perform well for new data (data not seen during training) → ***generalization***.

- Generalization refers to the ability of a ML model **to perform well** on unseen or new data **that was not used during training**.
    - It reflects how effectively the model has learned the underlying patterns in the data, **rather than memorizing** the training set.

ML Concepts

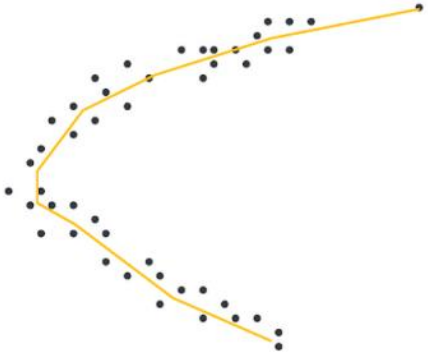
- Good Performance: What is good?
  - Different **peformance metrics** to measure **performance**
- Unseen data:
  - New examples from the same distribution as the training data → testing data
- Good generalization → two concepts:
  - **Overfitting**
  - **Underfitting**
- **Generalization error**: The generalization error is obtained measuring the performance of the model in the testing set.

Overfitting - Underfitting



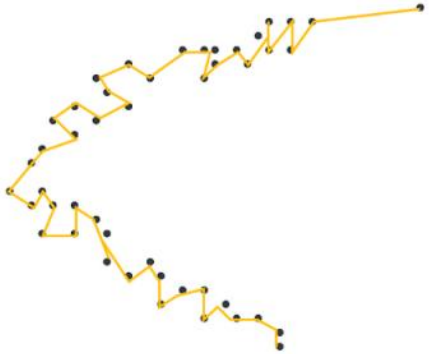
Underfitting

The ML **model is too simple** to capture the underlying relationship in the traning dataset.



Good Fit

Reduce the training error → small generalization error.



Overfitting

The model **fits the training data too closely, including noise**, leading to poor generalization on unseen data.

Image source: https://h2o.ai/wiki/overfitting/

Bias-Variance Trade-off

- **Bias**: $\left[f(x) - E\left[\hat{f}(x)\right]\right]$
  - Difference between the average prediction and the true function.
  - High bias leads to underfitting
- **Variance**: $E\left[\left(\hat{f}(x) - E\left[\hat{f}(x)\right]\right)^2\right]$
  - Error due to model sensitivity to small fluctuations in the training data.
  - High variance leads to overfitting

Goal: Find a balance between variance and bias.

Bias-Variance Tradeoff

- ↑ high bias → ↓ variance (⚠ underfitting)
- ↓ bias → ↑ variance (⚠ overfitting)
- **Model complexity** is a main factor → a model that is based on memorization is not able to predict correctly on unseen data ,

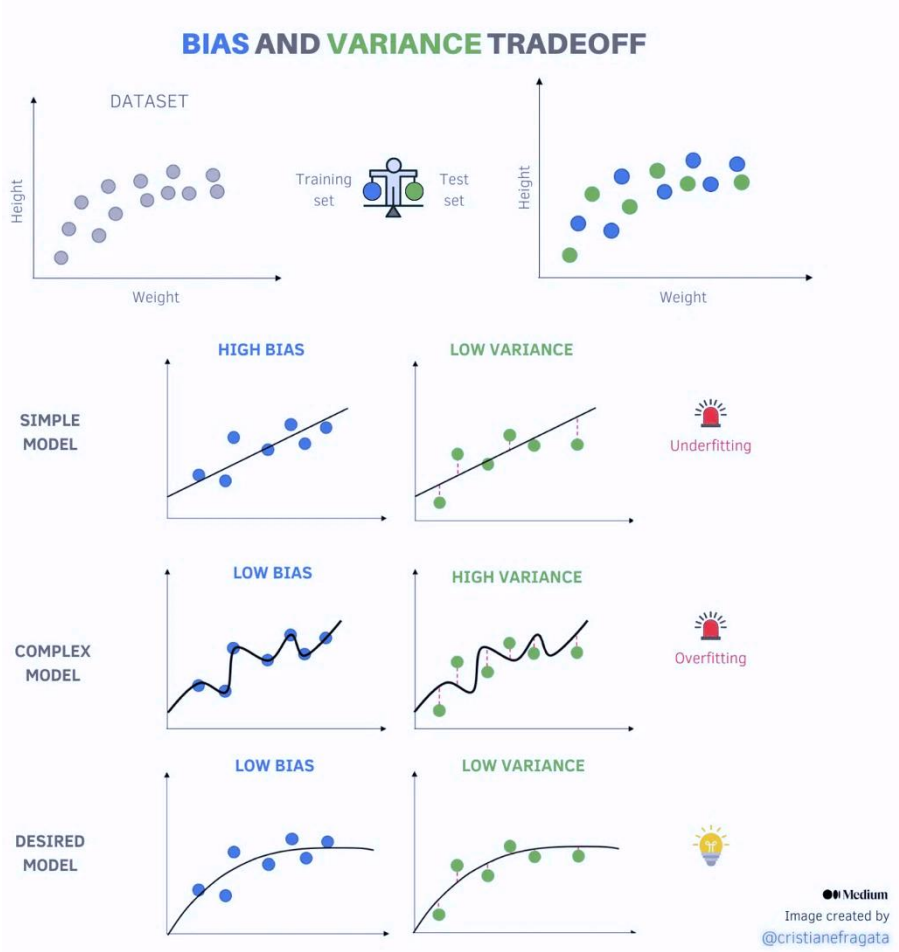
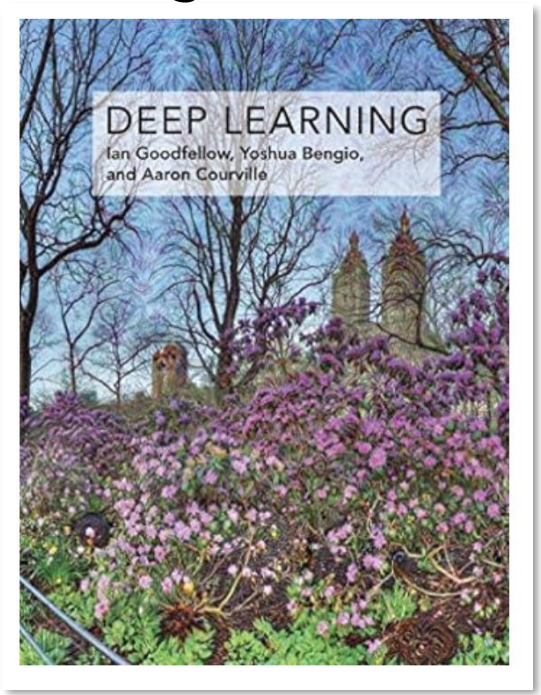
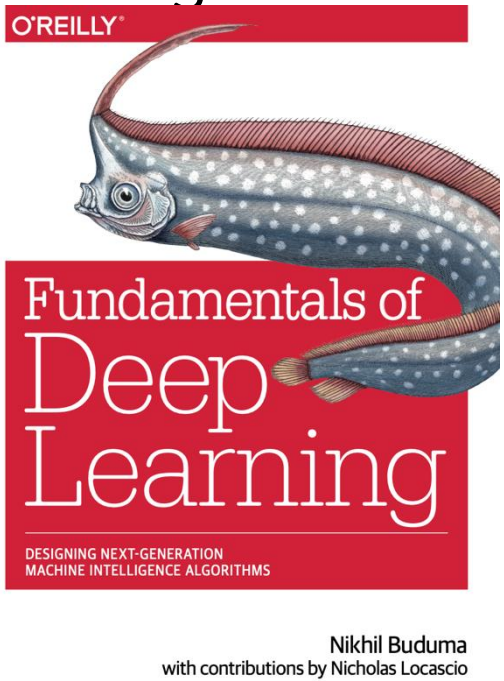


Image source: https://cristianefragata.medium.com/machine-learning-bias-and-variance-26b6ee572af

Deep Learning

Interesting Books to learn Deep Learning



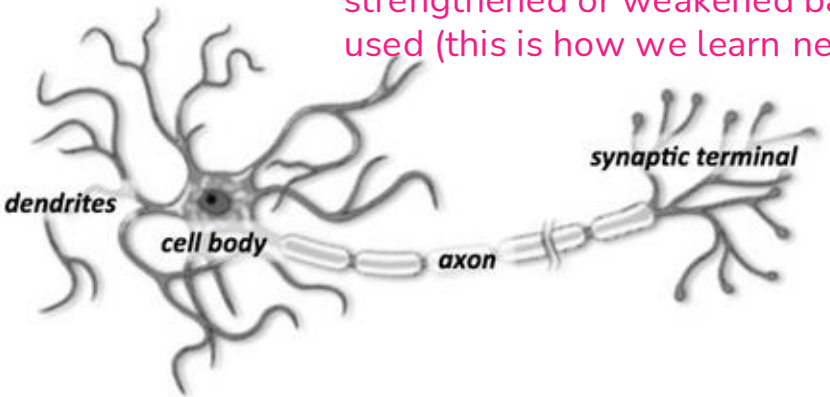


Online versión:
https://www.deeplearningbook.org

Introduction – Biological Neuron

(2) Each of the input signals is dynamically strengthened or weakened based on how often it is used (this is how we learn new concepts!)

(1) Dendrites receive the input signals



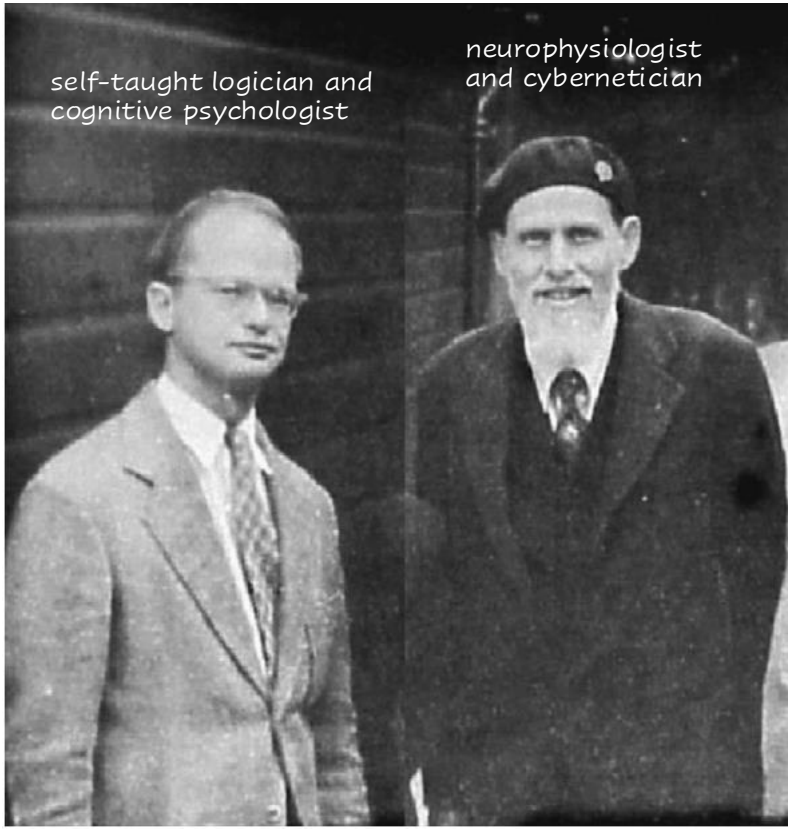
(3) After being weighted by the strength of their respective connections, the inputs are summed together in the cell body.

(4) This sum is then transformed into a new signal that's propagated along the cell's axon and sent off to other neurons.

Buduma, N., & Locascio, N. (2017). *Fundamentals of deep learning: Designing next-generation machine intelligence algorithms*. " O'Reilly Media, Inc.".

Artificial Neuron

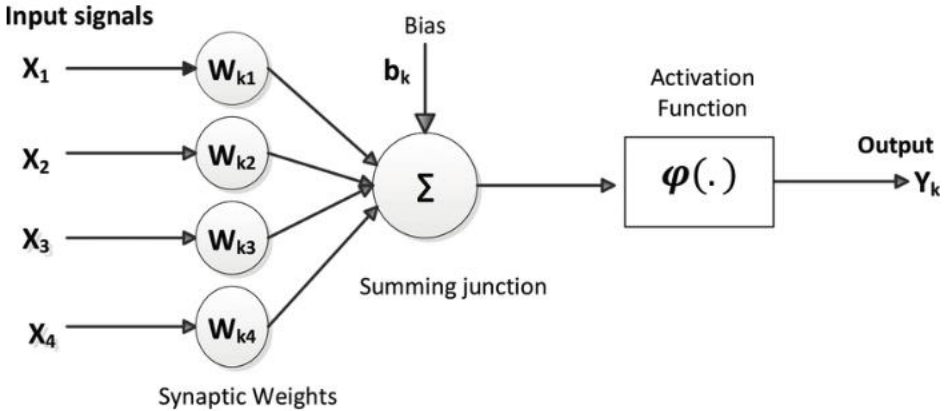
- The **artificial neuron** emerges from the behavior of the biological neuron (in a very simplified form).
- The first model of an artificial neuron was proposed by **McCulloch & Pitts** in 1943.



Warren Sturgis McCulloch (right) and Walter Harry Pitts (left) in 1949.
https://www.historyofinformation.com/detail.php?entryid=782

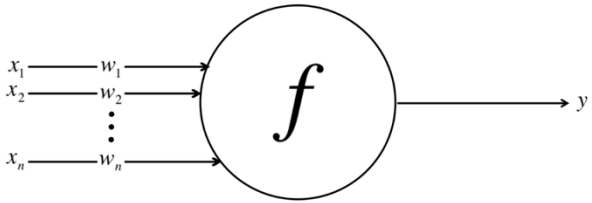
Artificial Neuron

- The artificial neuron receives inputs, and each input is multiplied by the weights.



Artificial Neural

- This weighted sum produces the **logit** of the neuron:



$$z = \sum_{i=0}^{n} w_i \, x_i$$

- The logit may include a **bias**, which corresponds to a constant value (not included in this figure).

- The **logit** is the input to the function to produce the output, and this output can be transmitted to other neurons.

Artificial Neural Network (ANN)

- Thus, an ANN is a non-linear function with many parameters.

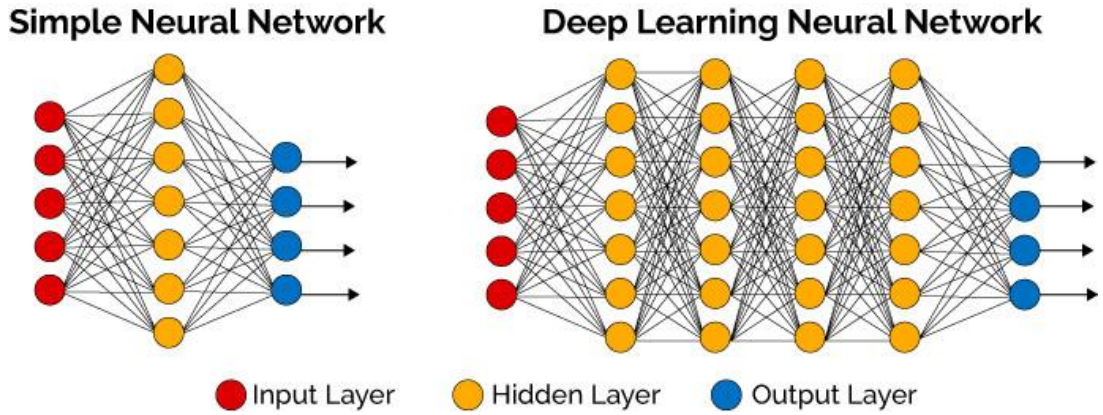


Image source: https://thedatascientist.com/what-deep-learning-is-and-isnt/

What is deep learning?

An artificial neural network with more than one hidden layer.

Feed-Forward Neural Network

- An FFNN is the simplest type of ANN where information flows in one direction: from input to hidden layers to output.

- How to train an ANN? → Minimizing the error between the actual and predicted value → as usual!

- How to measure the error? → Loss function

Loss functions

- In simple words is a function that compares the true and the predicted values, measuring how well the ANN models the training data.

- The loss function is minimized during training using optimization algorithms like the gradient descent.

- The choice of loss function depends on the type of problem and output format.
  - **Regression**: Use MSE or MAE (Mean Absolute Error).
  - **Binary Classification**: Use Binary Cross-Entropy.
  - **Multi-Class Classification**: Use Categorical Cross-Entropy.

Training

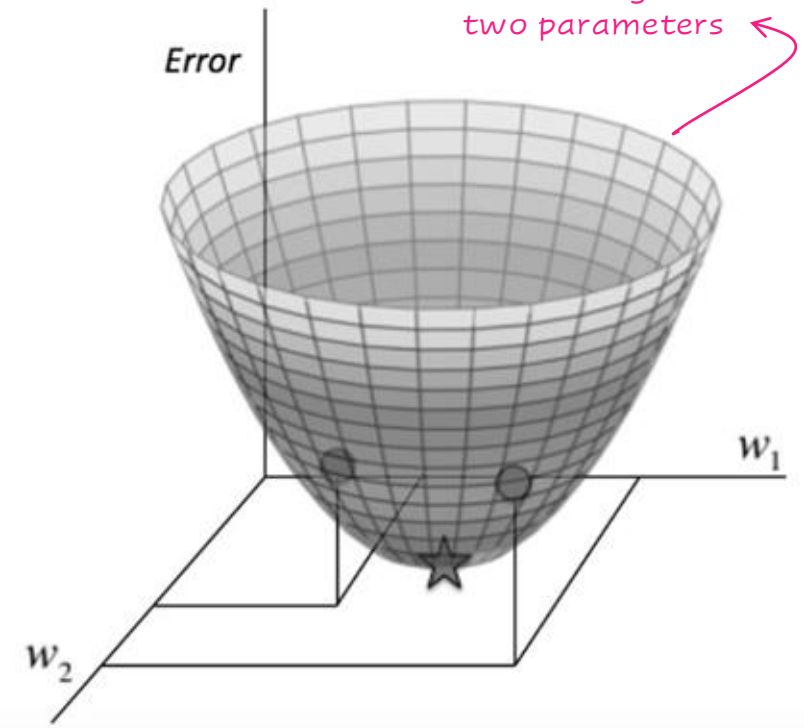
- To have an idea of the minimization process – the training – let's consider the following loss function(*):

$$E = \frac{1}{2}\sum_i (y_i - \hat{y}_i)^2,$$

A quadratic function

here, y_i is the actual value and $\hat{y}_i$ is the predicted value.

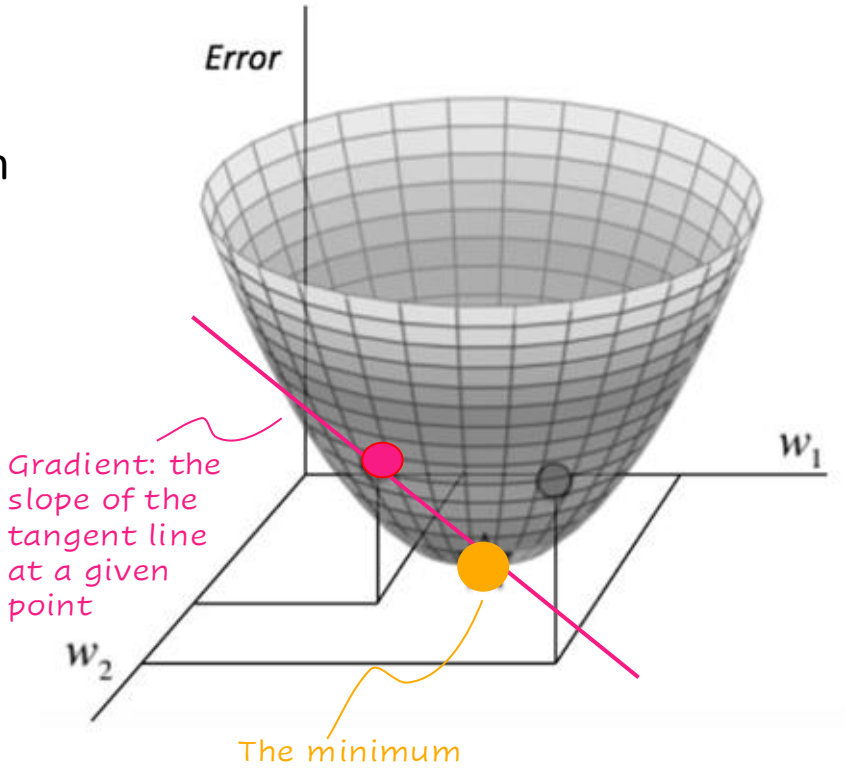
For simplicity, we are just considering a model with two parameters



(*) Using notation and explanation from: Buduma, N., & Locascio, N. (2017). *Fundamentals of deep learning: Designing next-generation machine intelligence algorithms*. " O'Reilly Media, Inc.".

Training

- **Minimize error** → optimization problem that seeks the parameter values that result in the lowest error.
- The gradient (**partial derivatives**) evaluated at a given point indicates the direction of the "steepest slope".
- The idea is to "move" along the surface to find the minimum.



(*) Using notation and explanation from: Buduma, N., & Locascio, N. (2017). *Fundamentals of deep learning: Designing next-generation machine intelligence algorithms*. " O'Reilly Media, Inc.".

Convolutional Neural Network (CNN)

- A neural network architecture is mainly used for visual tasks such as classification, object detection, segmentation, and more.
- FFNN cannot effectively capture the dependencies between pixels in an image because the image must be “flattened” into a one-dimensional array.
- CNNs take advantage of the multidimensional structure of images capturing the spatial relationships between pixels.

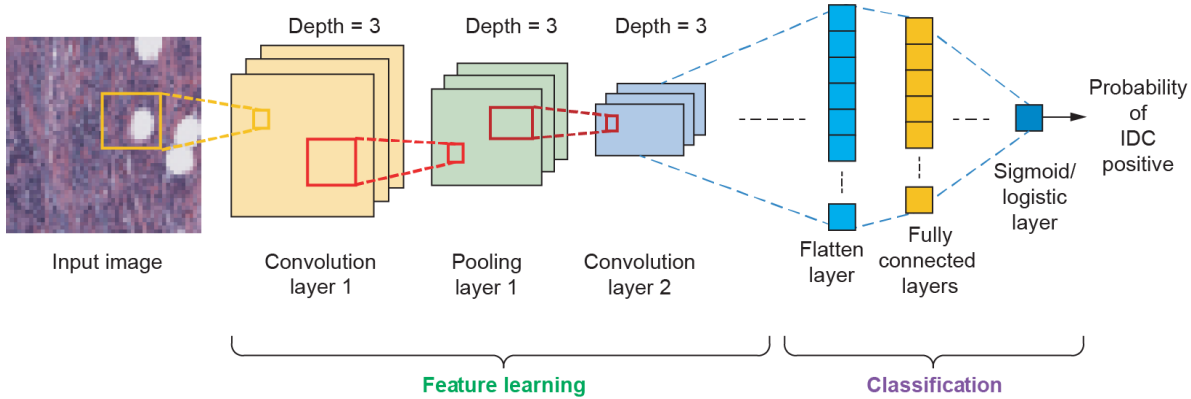
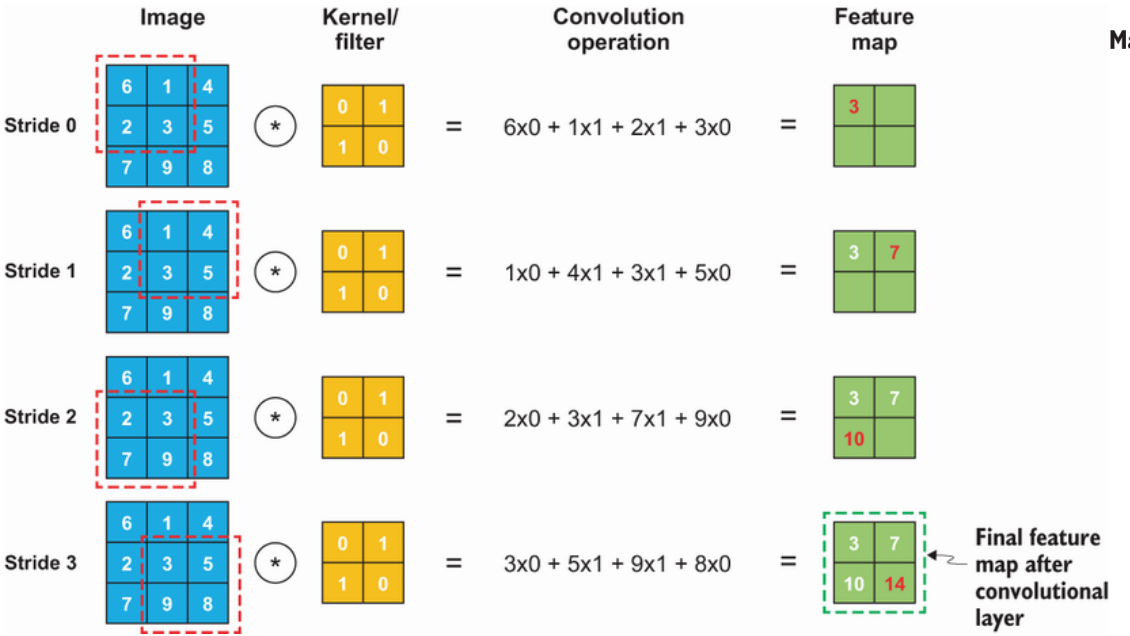
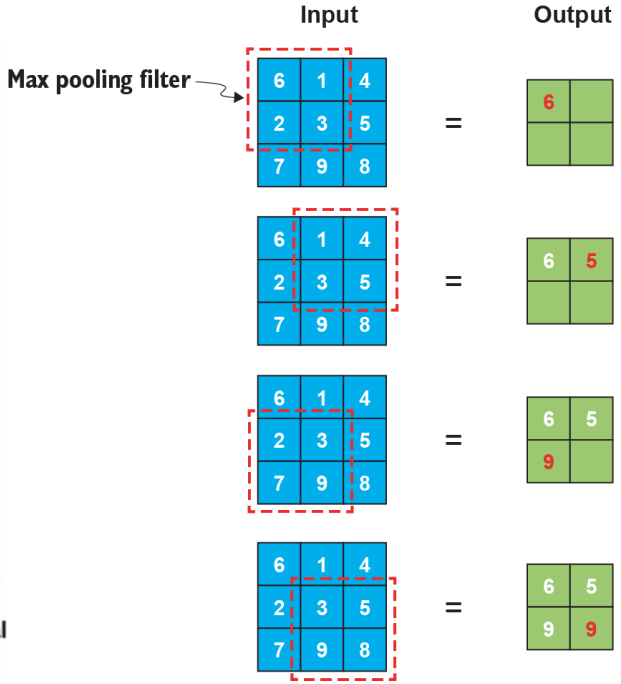


Image source: https://livebook.manning.com/book/interpretable-ai/chapter-5/25

Main CNN's layers



Convolutional layer



Max Pooling layers

Imagen obtenida de: https://livebook.manning.com/book/interpretable-ai/chapter-5/25

Convolutional layer

- Uses a **filter** or **kernel** whose elements are part of the parameters that define the CNN.

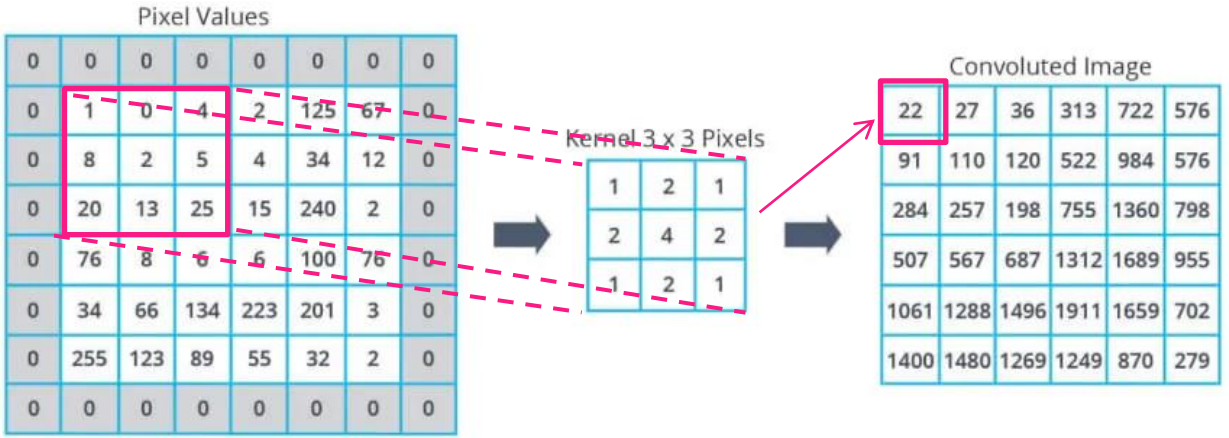
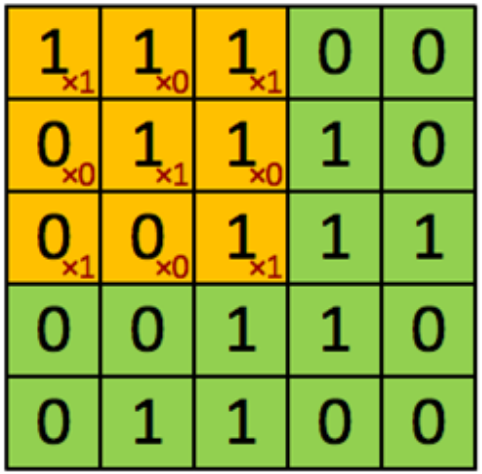
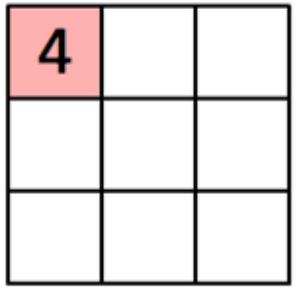


Image source: https://medium.com/@abhishekjainindore24/all-about-convolutions-kernels-features-in-cnn-c656616390a1

- **Filter size** (kernel size): The dimensions of the convolutional filter.
- **Number of output channels**: Specifies the depth of the **feature map** produced by the layer.
- **Stride**: The step size for sliding the kernel over the input.
- **Padding**: The method used to handle edge elements when the kernel does not fit perfectly.

Convolutional layer

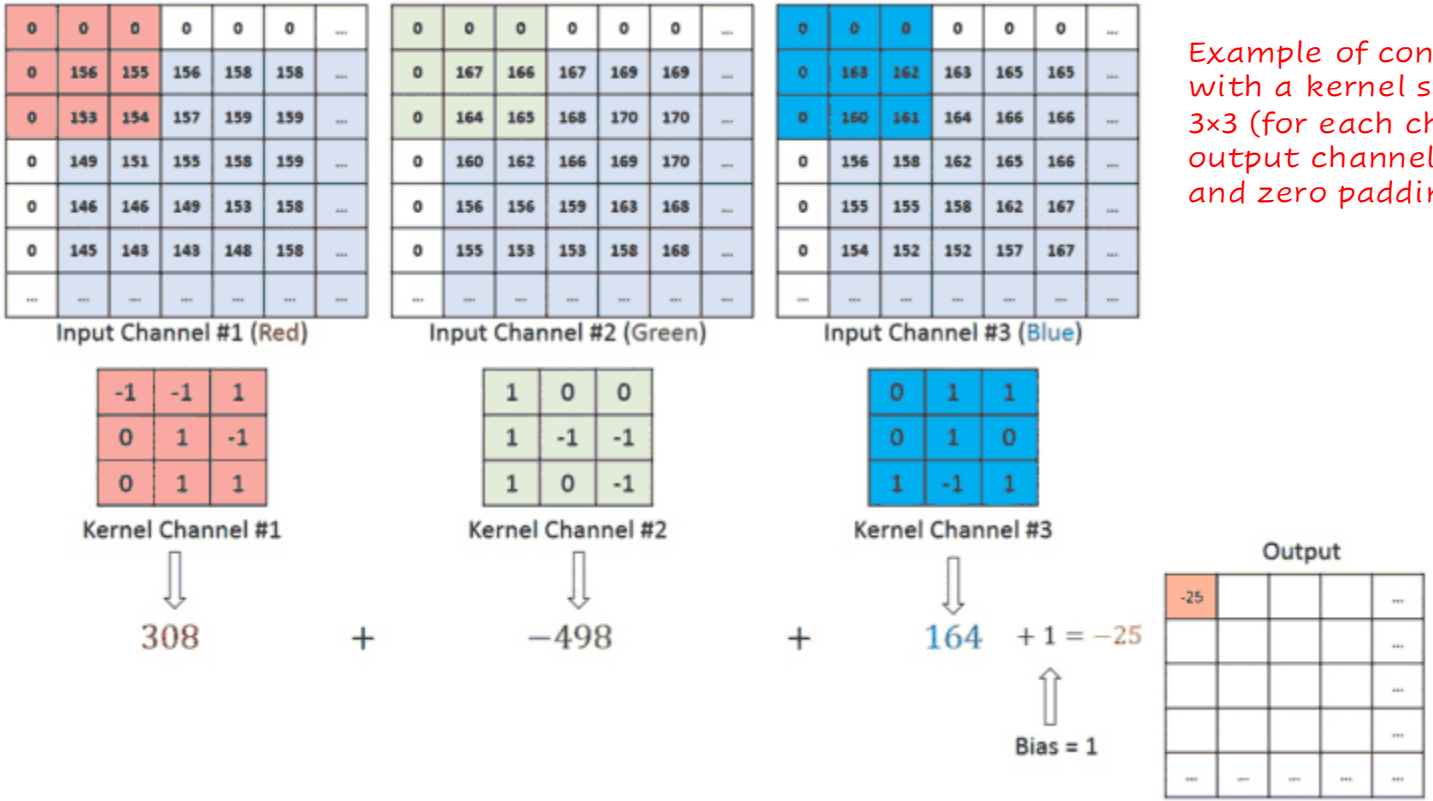




Convolution example with a 3x3 kernel, 1 output channel, stride=1 and without padding.

Image source: https://medium.com/nybles/a-brief-guide-to-convolutional-neural-network-cnn-642f47e88ed4

CNN's layers in a nutshell



Example of convolution with a kernel size of 3×3 (for each channel), 1 output channel, stride = 1, and zero padding.

Pooling layers

- Layers used to reduce the size of the input while extracting relevant features.
- Two main types of pooling:
  - **Max pooling** → Extracts the maximum value within the pooling window.
  - **Average pooling** → Extracts the average value within the pooling window.
- Key parameters include:
  - Filter size (pool size): The dimensions of the pooling window.
  - Stride: The step size for moving the pooling window across the input.
  - Padding: The method for handling border elements when the filter doesn't fit perfectly.

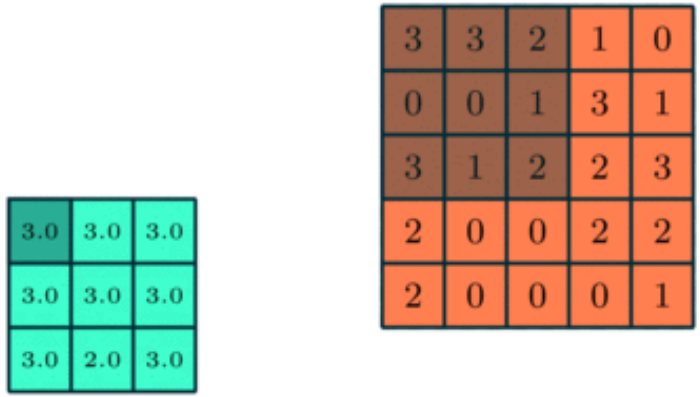


Image source: https://medium.com/nybles/a-brief-guide-to-convolutional-neural-network-cnn-642f47e88ed4

Fully Connected Layer

- A fully connected layer is simply a layer where **all neurons are connected to the neurons in the next layer**.

- The image is “flattened” to feed into the fully connected layer.
- This type of layer is useful, for example, for performing image classification tasks.

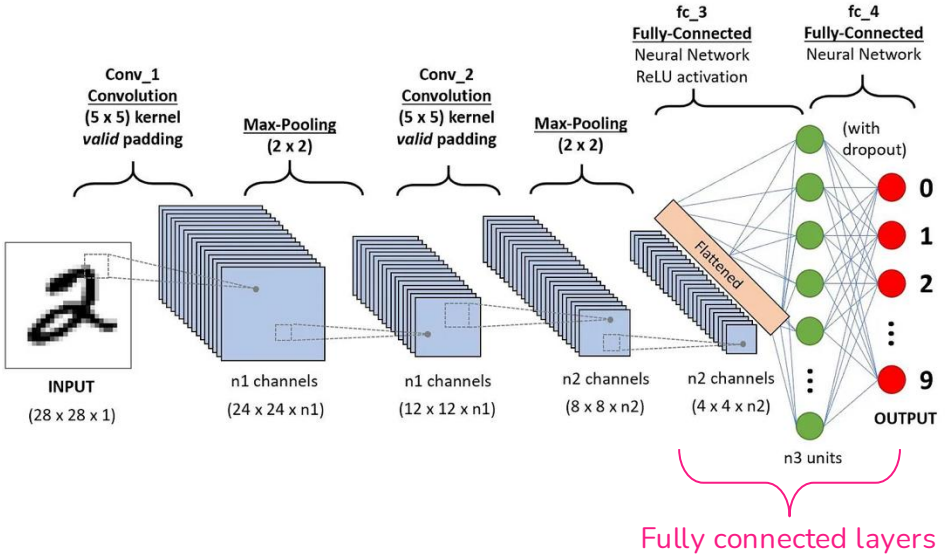


Image source https://medium.com/@alejandro.itoaramendia/convolutional-neural-networks-cnns-a-complete-guide-a803534a1930

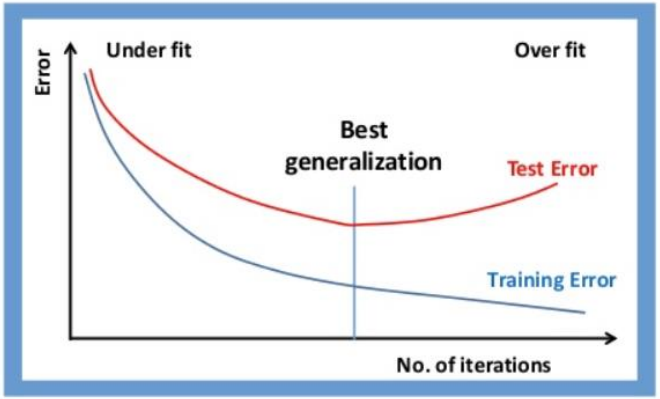
Introduction to Probabilistic NN

Introduction

- We typically work with deterministic models, which produce the same output for a given set of inputs every time they are executed.

- Model parameters are learned from data by minimizing the training error, with the goal of achieving a low generalization error on unseen data (test set).

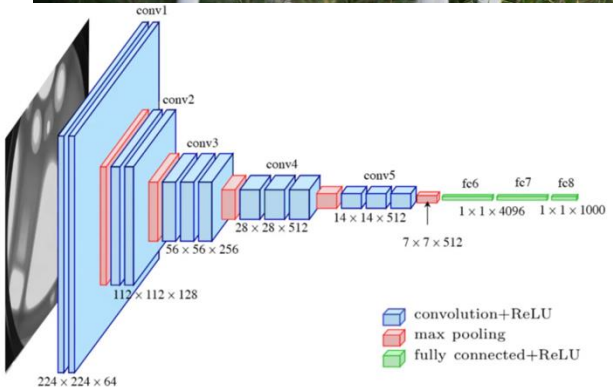
- However, this approach does not explicitly account for uncertainty estimation, which can be critical in many applications.



Introduction







VGG16 can classify with good performance those images.

Introduction

- What happens in this case?

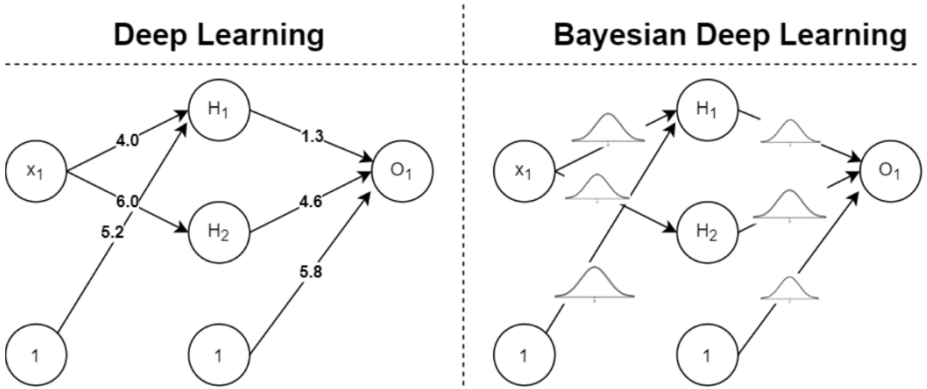


- The network cannot correctly classify the image because the training dataset does not contain images with elephants in an indoor environment.
- All training images show elephants in their natural habitat.
- We tipycally asume that:

$$P(\text{train}) = P(\text{test})$$

However, in practice, test data do not always follow the same distribution as training data.

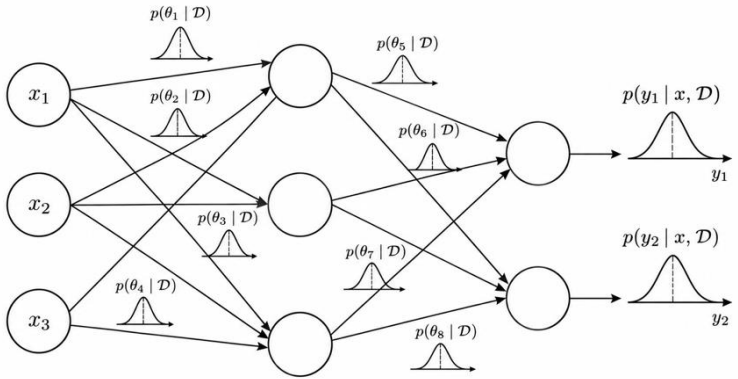
Bayesian Deep Learning (BDL) helps flag potentially incorrect predictions.



- Deterministic Neural Networks learn a single, fixed value for each weight (point estimates).
- Bayesian Neural Networks learn probability distributions over the weights instead of fixed values.
- These distributions allow us to capture uncertainty in the model parameters and incorporate it into predictions.

Bayesian NN Approach in a nutshell

Instead of a single set of weights, we seek a **probability distribution over the model parameters**.



Using **Bayes' theorem**, we obtain the posterior distribution:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)\ \ p(\theta)}{p(\mathcal{D})}$$

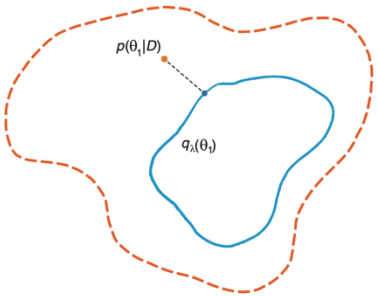
Challenge 1: Computing the exact posterior is generally intractable for deep neural networks.

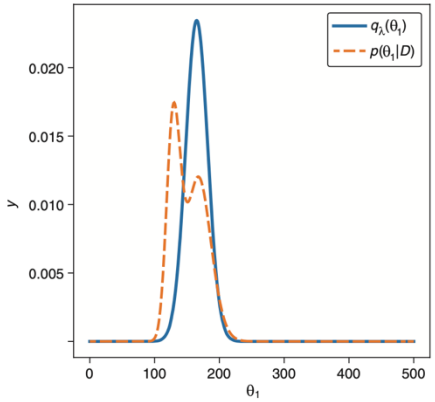
Bayesian NN Approach in a nutshell

- Our ultimate goal is to make predictions by integrating over all possible parameter values:

$$p(y|x_{\text{test}}, \mathcal{D}) = \int_{\theta} p(y|x_{\text{test}}, \theta)\, p(\theta|\mathcal{D}) d\theta$$

- **Challenge 2:** Computing the exact posterior predictive distribution is also generally intractable.
- **One solution:** Approximate Bayesian inference, such as Variational Inference (VI) and Monte Carlo Dropout (MC Dropout).

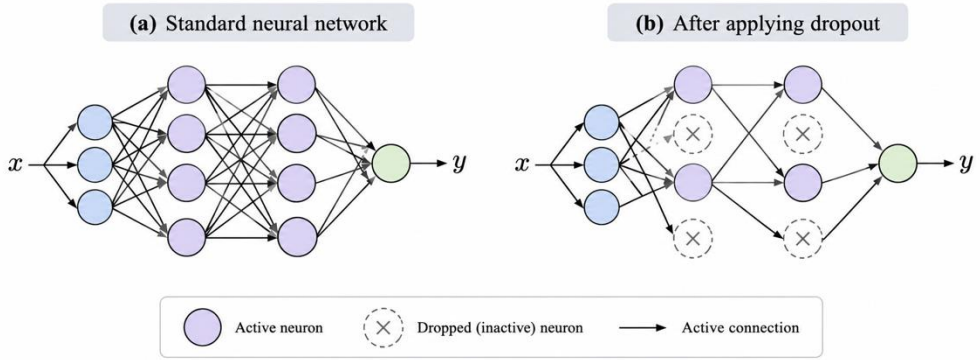




- VI approximates the posterior $p(\theta|\mathcal{D})$ with a simpler distribution $q_\lambda(\theta)$. However, this can still be computationally expensive.

MC Dropout as approximate Bayesian inference

- Dropout is a regularization technique that randomly deactivates neurons during training to reduce overfitting.

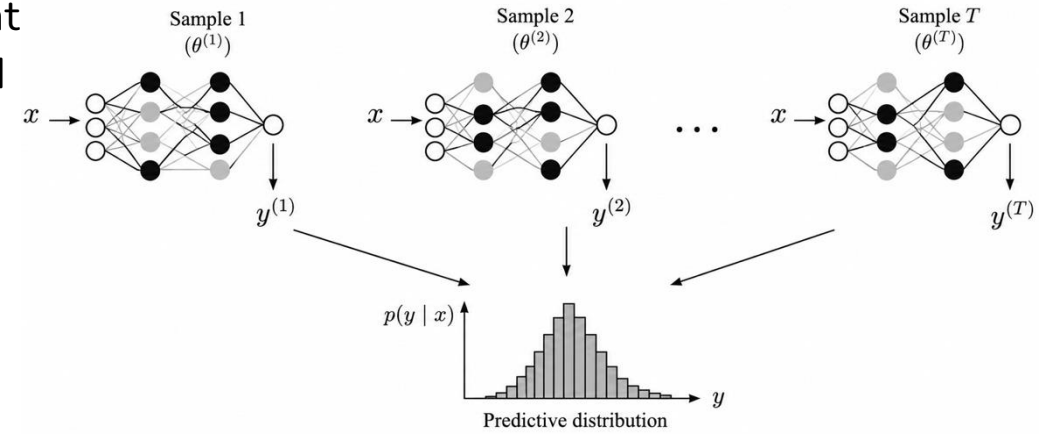


- This is achieved using Bernoulli-distributed binary masks (0 or 1)
  - which set effective weights to either 0 or their learned values θ_i, generating different network configurations.

- MC Dropout keeps dropout active during inference, producing multiple stochastic predictions to estimate uncertainty.

MC Dropout: Approximate Bayesian Inference

- Gal and Ghahramani (2016) showed that training with Dropout can be interpreted as approximate variational inference.



- The Bernoulli masks induce an approximate posterior distribution $q_\lambda(\theta)$ over the network weights.
- By keeping Dropout active during inference, each forward pass samples a different network configuration:

$$\theta^{(t)} \sim q_\lambda(\theta)$$

- Repeating this process T times generates multiple stochastic predictions:

$$p\left(y \middle| x_{test}, \theta^{(1)}\right), p\left(y \middle| x_{test}, \theta^{(2)}\right), \ldots, p\left(y \middle| x_{test}, \theta^{(T)}\right)$$

Uncertainty estimation

Having approximated the posterior predictive distribution, we can compute several uncertainty measures for regression tasks (such as stellar parameter estimation: $T_{eff}, \log g$, ...).

- Predictive mean: The average prediction across T stochastic forward passes.

$$\hat{\mu} = \frac{1}{T}\sum_{t=1}^{T} \hat{y}^{(t)}$$

- Predictive standard deviation: Measures the variability of the predictions, providing an estimate of epistemic uncertainty.

$$\hat{\sigma} = \sqrt{\frac{1}{T}\sum_{t=1}^{T} (\hat{y}^{(t)} - \hat{\mu})^2}$$

- Interquartile range (IQR): Measures the spread of the central 50% of predictions and is less sensitive to extreme values.

$$IQR = Q_{75} - Q_{25}$$

Uncertainty Estimation and Interquartile Interval

- Mean $\pm 2\,\sigma$

$$[\hat{\mu} - 2\hat{\sigma}, \hat{\mu} + 2\hat{\sigma}]$$

- 95% quantile Interval

$$[Q_{2.5}, Q_{97.5}]$$

- Intequartile Interval

$$[Q_{25}, Q_{75}]$$

Low uncertainty indicates consistent predictions, but does not necessarily imply high accuracy.

Sesión Práctica

A cargo de Bastián Wohlwend (USM)

https://github.com/bastifwp/SOCHIAS

Datos:
Daniela Turis (UV)

